WEEK 5

Payment Transaction Management System

DBMS Assignment Documentation

Submission contents

Payment table design, Order–Payment relationship, sample data, transaction management queries, payment analysis reports, and a query-output checklist.

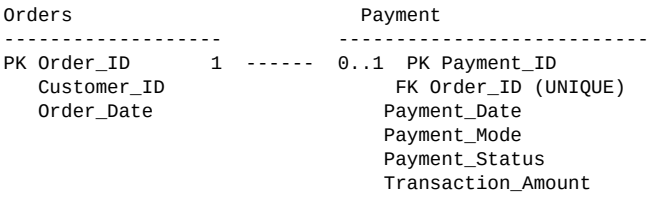
1. Payment Table Design

The Payment table stores payment transactions linked to existing Orders. Payment_ID is the primary key. Order_ID is both NOT NULL and UNIQUE, enforcing at most one payment row per order, and references Orders(Order_ID). CHECK constraints restrict payment modes, statuses, and non-negative transaction amounts.

Column	Suggested definition	Purpose
Payment_ID	INT PRIMARY KEY AUTO_INCREMENT	Unique payment identifier
Order_ID	INT NOT NULL UNIQUE	Links to Orders; max one payment per order
Payment_Date	DATE NOT NULL	Date of payment transaction
Payment_Mode	VARCHAR(30) NOT NULL	UPI/card/net banking/COD
Payment_Status	VARCHAR(20) NOT NULL DEFAULT 'Pending'	Pending, Success, or Failed
Transaction_Amount	DECIMAL(10,2) NOT NULL CHECK (>= 0)	Transaction amount

2. Updated ER Diagram (Order–Payment)

Relationship: one Order can have zero or one Payment record; each Payment record belongs to exactly one Order. The UNIQUE constraint on Payment.Order_ID enforces the maximum of one payment record per order.



3. Sample Customer, Order & Payment Data

The supplied SQL script inserts five sample payment rows for Order_ID values 1–5. It assumes the existing Customer and Orders tables already exist and contain matching records. Payment dates use CURDATE(), so the date is set when the script runs.

Sample payment rows (as defined in the SQL script):

Order_ID	Payment_Mode	Payment_Status	Transaction_Amount
1	UPI	Success	1300.00
2	Credit Card	Pending	2500.00
3	Debit Card	Failed → later updated to Success	800.00
4	Net Banking	Success	1500.00
5	Cash on Delivery	Pending	600.00

4. Payment Transaction & Revenue Analysis Reports

Successful payments: SELECT * FROM Payment WHERE Payment_Status = 'Success';

Failed and pending transactions: SELECT * FROM Payment WHERE Payment_Status IN ('Failed', 'Pending');

Transactions by payment mode: SELECT Payment_Mode, COUNT(*) AS Total_Transactions FROM Payment GROUP BY Payment_Mode;

Most preferred payment method: Group by Payment_Mode, count rows, and return the mode(s) with the maximum count.

Total successful revenue: SELECT SUM(Transaction_Amount) AS Total_Revenue FROM Payment WHERE Payment_Status = 'Success';

Revenue by payment method: SELECT Payment_Mode, SUM(Transaction_Amount) AS Revenue FROM Payment WHERE Payment_Status = 'Success' GROUP BY Payment_Mode;

Average transaction amount: SELECT AVG(Transaction_Amount) AS Average_Amount FROM Payment;

Customer payment history: Join Customer → Orders → Payment and sort by customer and payment date.

Status totals: Count Payment rows grouped by Payment_Status; optional sums can be included per status.

Orders with failed/pending payments: Join Orders and Payment, filtering status to Failed or Pending.

5. Query Output Screenshots

Screenshots must be captured after running the SQL script in MySQL Workbench or another SQL client connected to the database. Capture the query and result grid for: (1) Payment table rows, (2) successful payments, (3) failed/pending payments, (4) customer/order/payment join, (5) transaction count by mode, (6) successful revenue, and (7) status totals. Screenshots are not included in this PDF because no database execution output was supplied.

6. How to Run

1. Open the existing Order Management database. 2. Confirm Customer and Orders tables exist and the referenced columns match. 3. Run the table creation and INSERT statements from the accompanying SQL script. 4. Execute the report queries and capture screenshots. 5. Add screenshots to the final documentation before submitting to GitHub.

Important note

The supplied script depends on existing Orders and Customer tables and existing rows with Order_ID 1–5. If those tables or rows are missing, create/load the parent tables first or adjust the sample Order_ID values to match your database. CHECK constraint enforcement depends on the MySQL version.